

RESEARCH

Open Access



Real-time scheduling of power grid digital twin tasks in cloud via deep reinforcement learning

Daokun Qi¹, Xiaojuan Xi¹, Yake Tang^{1*}, Yuesong Zheng¹ and Zhengwei Guo¹

Abstract

As energy demand continues to grow, it is crucial to integrate advanced technologies into power grids for better reliability and efficiency. Digital Twin (DT) technology plays a key role in this by using data to monitor and predict real-time operations, significantly enhancing system efficiency. However, as the power grid expands and digitization accelerates, the data generated by the grid and the DT system grows exponentially. Effectively handling this massive data is crucial for leveraging DT technology. Traditional local computing faces challenges such as limited hardware resources and slow processing speeds. A viable solution is to offload tasks to the cloud, utilizing its powerful computational capabilities to support the stable operation of the power grid. To address the need, we propose GD-DRL, a task scheduling method based on Deep Reinforcement Learning (DRL). GD-DRL considers the characteristics of computational tasks from the power grid and DT system and uses a DRL agent to schedule tasks in real-time across different computing nodes, optimizing for processing time and cost. We evaluate our method against several established real-time scheduling techniques, including Deep Q-Network (DQN). Our experimental results show that the GD-DRL method outperforms existing strategies by reducing response time, lowering costs, and increasing success rates.

Keywords Digital twin, Power grid, Cloud computing, Task scheduling, Deep reinforcement learning

Introduction

With the rapid development of the global economy and the continuous improvement of people's living standards, energy demand shows a rapid growth trend [1]. The rapid growth of energy demand makes the stable operation of the power grid is facing unprecedented challenges. To improve the reliability and stability of the power grid, many cutting-edge technologies (e.g., smart grid, energy internet and artificial intelligence) have been carefully developed and integrated into power grid systems [2, 3]. Among these technologies, digital twin (DT) technology

has received focused attention due to its unique digital modeling and simulation capabilities [4].

Digital Twin technology, by creating virtual replicas of real-world systems, offers unprecedented insights and tools for the design, operation, and maintenance of power grids. DT can be used for forecasting operations and maintenance, load management, equipment failure prediction, and real-time power analysis [5, 6]. The key to realizing these functions is the processing of the large amount of data generated during the operation of the power grid and the DT system. For example, by recording the average operating temperature of transformer equipment in a power grid project and analyzing relevant historical data, we can estimate the transformer's average lifespan under current conditions. This offers valuable insights for operational and maintenance strategies. However, as the amount of equipment in power grid

*Correspondence:

Yake Tang

279527433@qq.com

¹ State Grid Henan Economic and Technological Research Institute, Zhengzhou, China

grows and data collection becomes more frequent, the volume of data increases exponentially. This renders traditional local data processing methods inadequate, posing a challenge in effectively managing the massive data.

The data produced by the power grid and its DT system necessitate extensive hardware resources for local computational processing, which incurs high computational costs and frequently falls short of real-time processing requirements. A new method for data management is essential, and the rapid advancements in cloud computing present a promising solution [7]. Specifically, tasks generated by the power grid and DT are offloaded to the cloud for processing. These computational tasks are executed using shared resources such as compute nodes, and the results are used for decision support. Figure 1 shows how tasks from both the physical and virtual entities are combined to form the DT, which is then uploaded to cloud computing nodes for processing. It also illustrates the typical architecture of a cloud computing environment used for power grid task processing. However, since the available computing nodes are limited, effectively scheduling these tasks to the appropriate nodes while ensuring quality of service (QoS) is a significant challenge that needs to be addressed.

In fact, task scheduling in cloud computing is a complex challenge that has gained extensive research in the field of optimization strategies. A large number of tools and algorithms have been developed and ongoing research aims to augment traditional approaches [8, 9].

However, popular strategies are primarily designed for batch tasks and cannot meet the demands of real-time workloads. With the advancement of artificial intelligence, learning-based methods have become a key research focus. Among these, reinforcement learning (RL) stands out as it can make real-time adjustments by learning from historical data and observed environmental information [10, 11]. Specifically, RL is a method for learning optimal strategies by having an agent interact with an environment, continuously trying and learning to maximize accumulated rewards. In scheduling problems, the agent can learn to dynamically allocate tasks to different resources or processing units within given constraints and objectives.

With the advances of deep learning, researchers are increasingly turning to Deep Reinforcement Learning (DRL) techniques [12, 13] to address task scheduling problems in cloud computing. Currently, cloud task scheduling using DRL combined with DT technology has been applied to various fields such as vehicle networking [14] and unmanned aerial vehicles [15], but there are few studies focused on power grids. To address the strict real-time requirements of power grid tasks and their DT systems, we propose a new scheduling method called GD-DRL. Our method can handle the massive tasks generated by the power grid and its DT system, providing real-time results to support subsequent decision-making. Specifically, given that the relevant tasks possess varying structural types and processing modes, GD-DRL utilizes

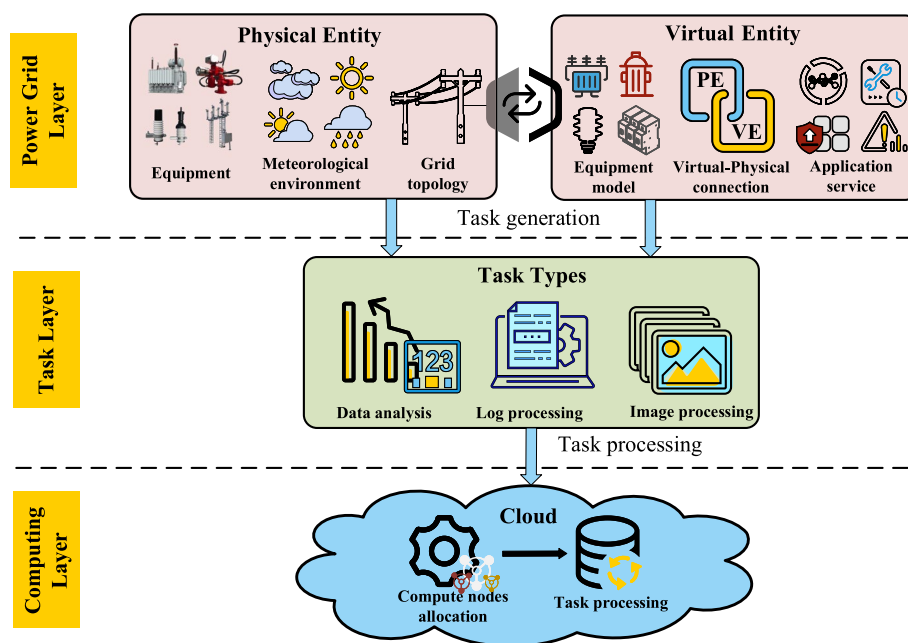


Fig. 1 Cloud computing with task processing for power grid digital twin

a DRL agent to allocate tasks to the most suitable computing nodes. It can not only minimize task response times but also reduce overall computational costs. In general, the main contributions of our work are summarized as follows:

- To address the limited research on using cloud computing to support power grids and DT systems, we propose a DRL-based task scheduling method that aims to minimize computational task response time while reducing overall computational costs.
- We provide a detailed mathematical model and implementation of our approach using Double Deep Q-Network.
- We compare our method with other commonly used real-time task scheduling approaches. Our experimental results show that our approach achieves better performance in terms of average task response time and success rate while significantly reducing execution costs.

The rest of this paper is organized as follows. Firstly, We review the relevant literature. Subsequently, We provide an overview of our proposed system architecture. Thereafter, We present the details of our DRL approach and evaluate the performance of our approach through experiments, followed by a conclusion summarizing our work.

Related work

DT creates precise digital representations of physical entities for purposes of simulation, analysis, and optimization. Initially introduced to support smart manufacturing within Industry 4.0 frameworks, DT leverages the synergies between information technology and physical production systems [16]. DT has been effectively employed for resource scheduling across various sectors, including smart cities, Telematics, and networks. For instance, the integration of DT techniques has been used to optimize offloading decisions and subchannel assignments, thereby improving computational rates and reducing task completion delays in these environments [17]. In the meantime, DT has also been used to solve problems arising in modern power systems. For example, the work [18] proposes a DT framework in smart grids to assess the remaining useful life of equipment. The successful application of DT in various domains also underscores their potential in revolutionizing cloud computing approaches within power grid. For example, DT-enabled cloud battery management systems have been proposed to augment computational and data storage capabilities in battery systems [19]. Additionally, a DT framework has been

proposed for the management of electrical devices, which ensures reliable device collaboration and efficient communication at the cloud edge [20]. DT technology has immense potential for power grid infrastructure, but managing the large volume of tasks it generates is crucial. Cloud computing is a better solution than local processing for handling these tasks. With limited computational resources, an efficient task-scheduling mechanism is essential to optimize resource use and maintain operational efficiency.

Task scheduling in the cloud is a long-standing problem and many solutions have been proposed. For example, researchers use the Whale Optimization Algorithm (WOA) to optimize task scheduling in cloud computing by improving traditional algorithms [21]. Another solution for task scheduling in cloud computing environments is the Enhanced Multi-Verse Optimizer (EMVO) algorithm [22], which can effectively reduce response time and improve resource utilization. Additionally, methods based on game theory [23, 24] and fuzzy logic [25] have been introduced to solve multi-objective optimization problems. However, most of these methods are designed for processing batch tasks and are not suitable for real-time workloads. For power grid, the importance of real-time is self-evident, so the focus of this paper is to address real-time task scheduling in cloud environments.

With the advancement of artificial intelligence technology, learning-based techniques don't need to construct models explicitly. They simply train neural networks based on historical data. The powerful perceptual ability of Deep Neural Networks (DNN) and the decision-making ability of Reinforcement Learning (RL) can effectively solve optimization problems [26]. Some researchers propose a collaborative MEC intelligent task offloading framework based on DT to develop a DRL-based Intelligent Task Offloading (INTO) scheme to jointly optimize peer-to-peer offloading and resource allocation decisions [27]. For DT-enabled edge computing vehicular networking, a DRL-based service offloading (SOL) method is proposed to compensate for the lack of vehicle computing resources in [28]. The DRL framework is used to optimize energy consumption as well as load balancing by leveraging DTs to aid in the deployment of Service Function Chain (SFC) over Computational Power Network (CPN) in [29]. Moreover, real-time data is collected using DT techniques and DRL algorithms are used to reduce the latency and energy consumption of task scheduling on the in-vehicle edge cloud [30]. DRL is then used to minimize the overall task completion delay and enable resource allocation. Furthermore, some people utilize DT to achieve smart distribution

network resource scheduling using deep Q-learning [31]. It is difficult for Q-learning to scale up to high-dimensional complex tasks and Deep Q-Network (DQN) [32, 33] can only suffer from the problem of Q-value overestimation.

Based on the above discussion, we summarize some of the main features of a typical task scheduling work, as shown in Table 1. Specifically, most existing work schedules tasks in batch form, while GD-DRL is able to schedule real-time tasks using Double Deep Q-Network (DDQN). We have applied DDQN to the task characteristics generated by the digital twin technology of the power grid, which is a novel contribution not previously reported in the literature. Moreover, few studies have considered the computational processing of tasks generated by power grid engineering and its DT system through the cloud. By scheduling appropriate compute nodes for the tasks, we reduce the response time and also reduce the computational cost. Furthermore, we have customized modifications to the DDQN algorithm based on the actual demands and characteristics of power grid scheduling, enhancing its suitability for handling the dynamic and real-time requirements of the power grid.

Architecture

In this section, we briefly describe the general system architecture of tasks generated by the power grid and DT system uploading to cloud computing nodes for processing and the associated mathematical models.

Figure 2 shows the task scheduling performed in the cloud computing nodes. When a task arrives, it is first sent to the queue of the desired instance selected by the scheduler. In the queue of the computing node, the task has to wait for its turn to be executed and once the execution is complete, a new task can be assigned to the computing node. To facilitate the description of the optimization problem studied in this paper, we provide mathematical definitions of the task model, computational node model, and task processing. The notations we used are shown in Table 2.

Task characteristics

With comparing tasks with other types of Digital Twin tasks in the cloud, the data generated by the power grid system comes from various sources, including sensors, measuring devices, monitoring systems, and others. Therefore, it is necessary to consider that the power grid and DT systems will generate various types of tasks

Table 1 Main features of typical approaches in literature

Ref.	Method	Real-time	Application system	QoS	Cost
[34]	HGSO	-	Cloud system	✓	✓
[22]	EMVO	-	Cloud system	✓	-
[35]	DQN	✓	Unmanned Aerial Vehicle	✓	✓
[28]	DQN	✓	Internet of Vehicles	✓	-
[36]	Q-learning	✓	Production control system	✓	✓
our approach	DDQN	✓	Power grid	✓	✓

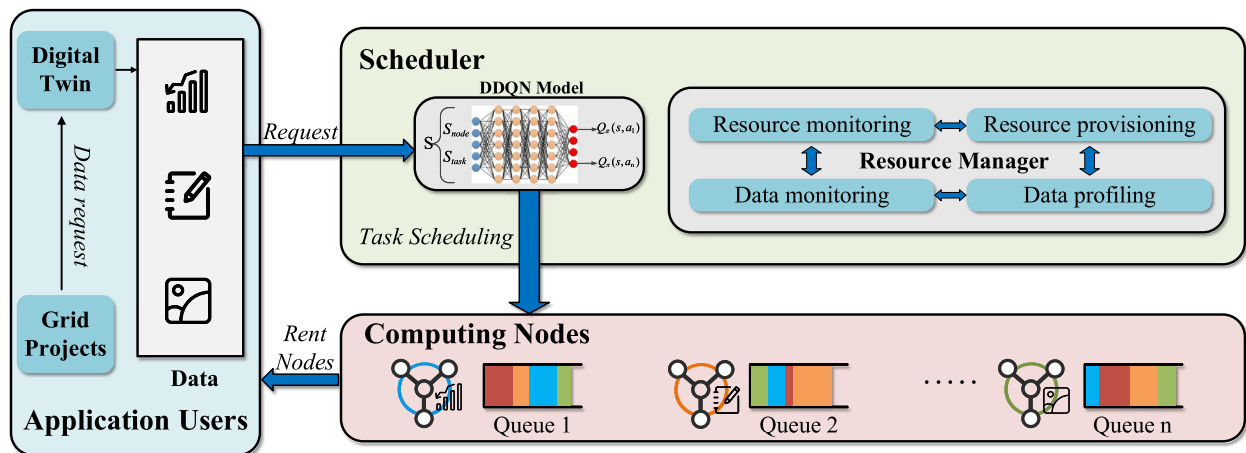


Fig. 2 The framework of task scheduling in cloud computing for power grid digital twin

Table 2 Notations used in our system

Notation	Meaning
D_i^{ID}	The id of the task
D_i^{AT}	The uploading time of the task
D_i^S	The size of the task
D_i^{QoS}	The QoS requirement for task processing
D_i^T	The type of the task
N_j^{ID}	The id of the computing node
N_j^T	The type of the computing node
N_j^C	The processing capacity of the computing node
N_j^P	The price for task processing
N_j^I	The idle time of the computing node
T_i	The response time of the task processing
T_i^{exe}	The execution time to process the task
T_i^{wait}	The waiting time for the task before processing
$cost_i$	The processing cost of the current task

such as data analysis, log processing, and image processing. Additionally, due to the highly dynamic nature of the power grid system, there is a high requirement for real-time responsiveness. Tasks need to promptly capture and reflect changes in the status of the power grid to support real-time monitoring, prediction, and decision-making. Consequently, we define tasks as uploadable at any time. For each uploaded task, we define the QoS time requirement. Each task is characterized as follows.

$$Task_i = \{D_i^{ID}, D_i^{AT}, D_i^S, D_i^{QoS}, D_i^T\} \quad (1)$$

where D_i^{ID} is the Task id, D_i^{AT} is the task uploading time, D_i^S is the size of the task, D_i^{QoS} is the QoS requirement for task processing, and D_i^T is the type of the task (i.e., data analysis, log processing, and image processing).

Computing nodes model

Computing nodes are the basic computing units in a cloud platform, and users have the flexibility to rent and manage these nodes according to their needs. For our problem, we assume a pay-as-you-go subscription for computing nodes. Each available computing node follows the following definition.

$$Node_j = \{N_j^{ID}, N_j^T, N_j^C, N_j^P, N_j^I\} \quad (2)$$

where N_j^{ID} is the id of the computing node, N_j^T is the type of the computing node (i.e., image processing, log processing, and data analysis), N_j^C is the processing capacity of the computing node, N_j^P is the price for task processing, which is related to the executing time. N_j^I is the idle time of the computing node.

Task scheduling model

We consider that as soon as the task is generated by the power grid and DT system, the scheduler sends it to the desired compute node. After scheduling, the task is added to the processing queue of the compute node containing all the tasks assigned to that compute node. This queue follows the first-come-first-served (FCFS) method to process the task. We assume that once a piece of task is being processed, no other task can interrupt its processing. We assume that there is no limit to the amount of tasks that can be added to the compute node queue.

In our problem, we define the response time for the task to complete processing as the sum of the time required to transform the task, the total time required to process the task, and the time the task spends in the computing node's waiting queue. Therefore, the task processing response time T_i is calculated as follows.

$$T_i = T_i^{wait} + T_i^{exe} \quad (3)$$

where T_i^{exe} is the time to process the task and T_i^{wait} is the waiting time for the task before processing. We further define the execution time T_i^{exe} .

$$T_i^{exe} = \alpha * \frac{D_i^S}{N_j^C}, \quad (4)$$

Where D_i^S is the size of the task, N_j^C is the processing capacity of the computing node. α is a constant parameter that denotes the speedup ratio of different types of tasks processed on different types of computing nodes. The specific values are shown in the Table 3.

When the task arrives and no task is being processed on the assigned computing node, it is executed immediately. Otherwise, it needs to wait for the current task processing to finish before proceeding to the next operation. Further, We define the waiting time T_i^{wait} as follows.

$$T_i^{wait} = \begin{cases} N_j^I - D_i^{AT} & \text{if } N_j^I > D_i^{AT} \\ 0 & \text{otherwise} \end{cases} \quad (5)$$

where N_j^I is the idle time of the computing node, D_i^{AT} is the task uploading time. Therefore, if the computing node is not idle, the waiting time for task processing is the computing node's idle time minus the task uploading

Table 3 The value of α

N_j^T	α	D_i^T	image processing	log processing	data analysis
image processing	1			1.2	1.1
log processing	1.5			1	1.3
data analysis	1.7			1.4	1

time. Conversely, if the computing node is idle, no wait is required.

Task processing is considered successful if the QoS requirement is satisfied, so the condition for successful task processing can be defined as below.

$$success = \begin{cases} 1 & \text{if } T_i < D_i^{QoS} \\ 0 & \text{else} \end{cases} \quad (6)$$

where T_i is the task processing response time, D_i^{QoS} is the QoS requirement for task processing.

The cost of task processing depends on the execution time. The shorter the execution time of each task processing task, the lower its cost. The cost of each task processing can be defined as follows.

$$cost_i = N_j^P * T_i^{exe} \quad (7)$$

where $cost_i$ is processing cost of the current task, N_j^P is price for task processing of the computing node, T_i^{exe} is the time to process the task.

Approach

We focus on utilizing the DRL approach to solve the task scheduling problem of uploading tasks to cloud nodes for processing. This section describes the fundamentals of the DRL and the framework of GD-DRL.

Basics of deep reinforcement learning

Markov Decision Process and Q-learning: Markov Decision Process (MDP) is widely used to provide a mathematical framework [37] for modeling decision making to solve stochastic sequential decision problems, in situations where an outcome is partly random and partly under the control of the decision maker. MDPs have been a useful approach for studying a wide range of optimization problems solved via dynamic programming and reinforcement learning. A Markov decision process is a 5-tuple $(S, A, P_a(s, s'), R_a(s, s'), \text{ and } \gamma)$. The specific definitions are as follows.

- S is a finite set of states;
- A is a finite set of actions;
- $P_a(s, s') = Pr(s_{t+1} = s' | s_t = s, a_t = a)$ is the probability that action a in state s_t at time t will lead to state s_{t+1} at time $t + 1$;
- $R_a(s, s')$ is the immediate reward (or expected immediate reward) received after transitioning from state s_t to state s_{t+1} , due to action a ;
- $\gamma \in [0, 1]$ is the discount factor, which represents the difference in importance between future rewards and present rewards.

The MDP's goal is to find an optimal policy that will maximize the expected return from a sequence of actions that leads to a sequence of states. The expected return can be defined as a policy function that calculates the sum of the discounted rewards. The goal of finitely satisfying the Markov property in reinforcement learning is to form a state transition matrix that consists of each possibility of transitioning to the next state from a given state.

Q-learning [38] is a model-less reinforcement learning technique, and its implementation is similar to the MDP strategy. Specifically, Q-learning can be used to find the best behavior choice for any given (finite) MDP. The working principle of Q-learning is to learn an action value function and give the final expected result. The Q-value is the cumulative return value obtained after executing action a in state s . The algorithm constructs a Q-table to store the Q-values, facilitating the selection of actions that optimize returns. Decision-making relies solely on comparing the Q-values for each possible action in a given state s , without considering the subsequent states of s . The Q-value is updated based on observed rewards and states, influenced by a learning rate α and a discount factor γ . The iterative process ultimately leads to the convergence of the Q-value to $Q(s, a)$, delineating the optimal action value function. The values in the Q-function are updated using the following expression.

$$Q(s_t, a_t) = Q(s_t, a_t) + \alpha * [r_t + \gamma * \max_a Q(s_{t+1}, a) - Q(s_t, a_t)] \quad (8)$$

DQN: The traditional Q-learning algorithm works well when dealing with finite discrete state spaces, but encounters challenges in high-dimensional continuous state spaces. This is because it relies on a Q-table to store the value of each state-action pair, and in high-dimensional spaces, this table can become extremely large and difficult to manage and update. DQN [39] overcomes this limitation by approximating the Q-value function using a deep neural network, where the weights of the network are denoted by θ handles complicated decision making with large and continuous state space. DQN takes state features as raw data input and output the Q function value of each state-action pair. It can learn mappings from states to action values without the need for a huge Q-table.

DQN uses goal networks and experience playback to improve stability and convergence. Empirical playback is a key feature of DQN that allows the algorithm to store past transitions (s_t, a_t, r_t, s_{t+1}) and use this data multiple times during training. This approach breaks the temporal correlation between data and makes more efficient use of the data. Fixed Q-target [40] is another improvement,

which involves using a fixed network to generate the Q-target value during the update step instead of using the main network. This helps to stabilize the learning process as it reduces the correlation between targets and predictions. In the DQN algorithm, the objective function is defined as below.

$$Y_t^{DQN} = r_{t+1} + \gamma \max_a Q(s_{t+1}, a; \theta_t) \quad (9)$$

DDQN: Despite its contributions to the Q value to approach the optimization goal quickly, the greedy method is also prone to overfitting. In the DQN model, the target network adopts the action selection strategy of maximizing the Q value by $\max_a Q(s_{t+1}, a; \theta_t)$. Therefore, the algorithm model obtained by DQN probably has a large deviation, which results in overestimation. To tackle the overestimation problem, we need to change the training method of the network self-fitting. Therefore, this study proposes a learning model based on DDQN [41] to modify the calculation method for task scheduling. By placing the calculation and action selection of the target Q value in two networks, the transfer of the maximum deviation is cut off.

In DDQN, to minimize the problem of overestimation, the computation of the target Q-value is divided into two steps. First, the online network is used to select the best action and then the target network is used to compute

the Q-value of this action. The objective function in DDQN is adapted as below.

$$Y_t^{DDQN} = r_{t+1} + \gamma Q(s_{t+1}, \arg \max_a Q(s_{t+1}, a; \theta_t), \theta'_t) \quad (10)$$

The agent receives the state of virtual machines and tasks from the edge scheduling environment. Then appropriate actions, the task-to-computing node mappings, are generated by the evaluation network. Then the agent assesses whether all the tasks have been scheduled. If scheduling is complete, then the target value is r_{t+1} . If not, the target network is used to calculate the target value. The gradient descent algorithm is then performed to update the evaluation network.

The proposed GD-DRL framework

Algorithm 1 gives the details of the GD-DRL based task scheduling cost optimization algorithm. The algorithm determines where tasks should be queued for execution. Specifically, our algorithm checks the current state of all available nodes by using q-values to determine the choice of compute nodes. In parallel with this task scheduler, we also keep track of the task queue of the compute nodes, updating it when a new task arrives or when task processing is complete. A more detailed mathematical model defining states and rewards in detail will be mentioned in a later section.

Algorithm 1 GD-DRL task scheduling cost optimization algorithm

```

Initialize  $\epsilon, \alpha, \gamma$ , target network update frequency  $C$ , start learning time  $\tau$ , minibatch size  $M$ ,
replay memory  $D$ 
Initialize action-value function  $Q$  and target value function  $Q'$  with random weights
for each episode do
  Initialize state  $s_t$ 
  for each step of episode do
    With probability  $\epsilon$  select a random action (computing node)  $a$ , otherwise select
     $a = \arg \max_a Q(s, a; \theta)$ 
    Add the task to the node queue; Execute action  $a$ , observe reward  $r$  and new state
     $s_{t+1}$ 
    Store transition  $(s_t, a_t, r_t, s_{t+1})$  in  $D$ 
     $s_t \leftarrow s_{t+1}$ 
    if learning starts after  $\tau$  steps then
      Sample a minibatch of transitions  $(s_t^j, a_t^j, r_t^j, s_{t+1}^j)$  from  $D$ 
      if  $s_{t+1}^j$  is a terminal state then
        Set  $Y_t^j = r_t^j$ 
      end
      else
         $Y_t^j = r_t^j + \gamma Q'(s_{t+1}^j, \arg \max_a Q(s_{t+1}^j, a^j, \theta_t); \theta'_t)$ 
      end
      Perform a gradient descent step on  $(Y_t^j - Q(s_t^j, a^j; \theta_t))^2$  with respect to the
      network parameters  $\theta_t$ 
    end
    Every  $C$  steps update the target network parameters  $\theta'_t = \theta_t$ 
  end
end

```

Action Space: We assume that a fixed number of cloud computing nodes are available. These computing nodes have a task queue in which a task is allocated by the task scheduler. Once the tasks are added to the queue, they will be executed in FCFS fashion. Therefore, we define the action space as the set of all computing nodes available. Thus, our action space is defined as:

$$a = \{N_1, N_2, \dots, N_m\} \quad (11)$$

where m is the number of computing nodes available to process the task.

State Space: We assume that the new task is ready to be scheduled at time t , and we can define the state space of DDQN at time t as:

$$S = S_{node} \cup S_{task} \quad (12)$$

where S_{node} and S_{data} are the states of the computing nodes and the current task at time t , respectively. More specifically, the entire state space can be described as:

$$S = \{AT_t, T_t, N_1^t, N_2^t, \dots, N_m^t\} \quad (13)$$

where AT_t and T_t denote the uploading time and type of the current task. And N_1^t signifies the wait time for the task in N_{th} computing node.

Reward function: Since the task processing process has to satisfy the QoS requirements as well as minimize the response time and cost, we consider that the effectiveness of the cost per task and the average response time affects the reward. Therefore, the reward function of DDQN is defined as:

$$r = \begin{cases} -(\lambda_1 * T_i + \lambda_2 * cost_i) & \text{if success} = 1 \\ -\lambda_3 & \text{else} \end{cases} \quad (14)$$

where T_i and $cost_i$ indicate the response time and cost of the current task, with smaller values yielding higher rewards. Moreover, λ_i are the trade-off parameters utilized to coordinate the effect of cost and response time. Increasing λ_2 emphasizes response time, while a higher λ_1 value suits cost prioritization. Additionally, it is worth noting that $\lambda_1 + \lambda_2 = 1$, and λ_3 is set to constant.

Model Training Period: During the model training period, decisions and outcomes from historical scheduling are leveraged to guide the underlying DNN to acquire a more accurate value function. The training procedure of the DDQN scheduling model is illustrated in Algorithm 1. Before training begins, we first initialize the parameters of the algorithm, including the exploration rate ϵ , the learning rate α , the discount factor γ , the target network update frequency C , the learning start time τ , the small batch size M , and the playback memory bank D . Then, we randomly initialize the weights of the action-valued function Q and the target-valued function Q' .

In each step of every episode, the agent selects an action based on the current state. Specifically, the agent selects a random action with probability ϵ which means randomly choosing one among the available computing nodes. Otherwise, it selects the action that maximizes $Q(s, a; \theta)$. After selecting an action, the agent adds the task to the corresponding node queue, executes the selected action, observes the reward and the new state, and stores the transition information in the replay memory. If the number of steps after learning starts exceeds τ , the agent samples a minibatch of transitions from the replay memory and utilizes these transitions to perform gradient descent for updating the weights of the action-value function. Additionally, we periodically update the parameters of the target network to stabilize the training process and improve the performance of the algorithm.

In the algorithm, we utilize an ϵ -greedy strategy to balance exploration and exploitation. Specifically, with a probability of ϵ , a random action is chosen to explore the environment, while with the remaining probability $(1 - \epsilon)$, the action that maximizes the action-value function is selected to exploit the known information. This balance allows the algorithm to continuously explore new task scheduling strategies during the learning process and gradually iterate towards utilizing the optimal strategy. Additionally, we employ the technique of experience replay to train the agent. This involves utilizing past experiences to smooth the training process, reduce correlations between samples, and enhance the efficiency and stability of training.

Evaluation

In this section, we present the experimental evaluation of our proposed method. Firstly, the experimental setup is given, including the configuration of the experimental environment and the hyperparameters of the DRL algorithm. Then, the results for different workload scenarios are presented.

Experimental settings

We configured three groups of different types of cloud computing nodes. Table 4 contains detailed information about these computing nodes. These nodes of different types have varying computational capabilities and costs.

Table 4 Computing nodes types used in our evaluation

Computing node	Processing capacity	Processing price
Image processing	1500	4
Log processing	1200	2
Data analysis	1000	1

In the experiment, we utilized the following hyperparameter settings for our method: the total number of training episodes was set to 1,000, with a target network update frequency of 20. The size of the replay buffer was set to 100,000, and the batch size was chosen as 256. The learning rate and discount factor were respectively set to 0.001 and 0.999. We compared our method against four common scheduling methods (Random, Round-Robin, Earliest, and DQN). We implemented the proposed GD-DRL method using the PyTorch framework and conducted training and inference on a GPU. The hyperparameters for the GD-DRL in this experiment are presented in Table 5.

Experimental results

In this part, we carried out experiments with task sizes, task arrival rates, task ratios, and number of compute nodes to cover a wide range of scenarios.

Varying the Size of Tasks: In this experiment, we compared the performance of different methods in handling various average task sizes. The experiment involved setting different average task sizes from 200 to 1000. The proportions of data analysis, log processing, and image processing in the workload were set to 0.5, 0.3, and 0.2, respectively. According to the results shown in Fig. 3,

GD-DRL outperformed other algorithms in task scheduling. Specifically, regardless of task size, GD-DRL exhibited lower average response times and costs, while also significantly surpassing other algorithms in success rate.

It is worth noting that although the response times of DQN and DDQN increased with the increase in task size, they still managed to maintain relatively low response times when dealing with large-scale tasks. This highlights the potential of deep reinforcement learning algorithms in handling large datasets. Particularly with DDQN, its lower response time indicates effective mitigation of the overestimation problem, giving it an advantage in handling large-scale tasks, and further reinforcing the performance of GD-DRL.

Overall, the increase in task size leading to higher costs is expected, as handling larger tasks naturally requires more resources. Simultaneously, the decrease in success rates is also anticipated, as larger data may cause resource bottlenecks, affecting task completion rates. In this regard, GD-DRL performed well, with its stable success rate surpassing other algorithms, demonstrating its reliability and efficiency in task scheduling.

Varying the Arrival Rate of Tasks: In this experiment, we compared the performance of different methods under varying task arrival rates. The experiment set different average data arrival rates ranging from 10 to 30. According to the results shown in Fig. 4, GD-DRL outperformed other algorithms in task processing. Specifically, regardless of the average data arrival rate, GD-DRL consistently exhibited lower average response time and cost, while also demonstrating significantly higher success rates compared to other algorithms.

Regarding average response time, Random and Round-Robin showed higher response times, while DQN and DDQN methods maintained relatively lower levels. Particularly, the DDQN strategy maintained lower response

Table 5 The hyperparameters of GD-DRL

Hyperparameters	Value
Learning rate	0.001
Discount factor	0.999
Batch size	256
Maximum episode	1000
Replay buffer	100,000
Frequency of target network updates	20

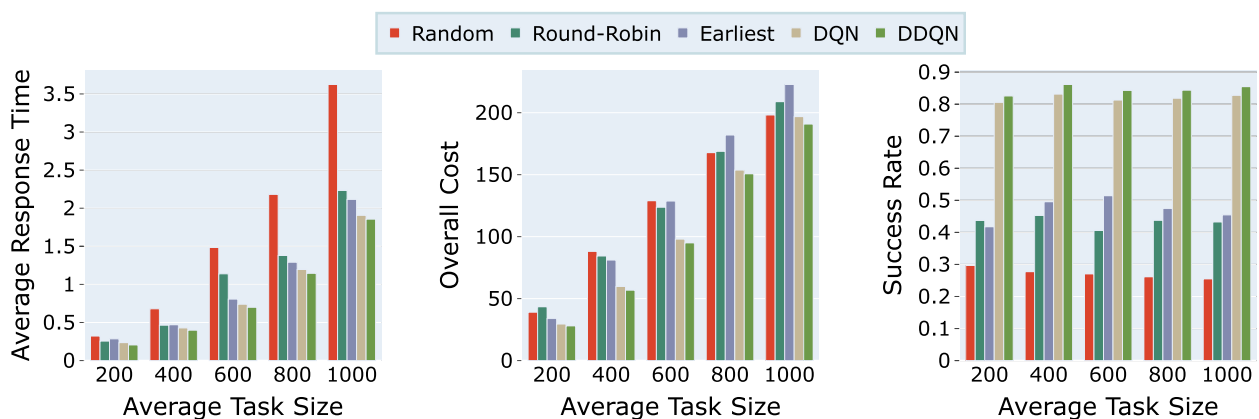


Fig. 3 Performance under different task sizes

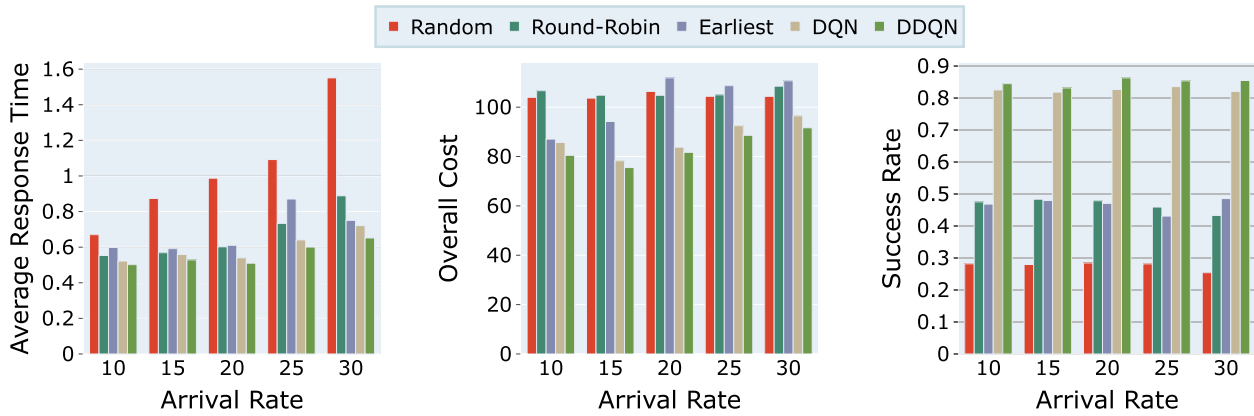


Fig. 4 Performance under different arrival rates

times even at higher arrival rates, demonstrating stable performance. In overall cost evaluation, the costs of all methods increased with the increase in arrival rates. However, DDQN exhibited better performance in cost control, with a smaller increase in costs even at high arrival rates, reflecting its advantage in resource utilization efficiency. For success rate, DDQN maintained higher success rates under all arrival rate conditions, while other methods gradually decreased with increasing arrival rates. This result suggests that DDQN has stronger stability and reliability in ensuring successful task completion. For the results, we can see that DDQN performed excellently in all performance metrics, particularly in handling tasks with a high arrival rate, effectively balancing response time, cost, and success rate. This demonstrates the superiority of our proposed GD-DRL in complex data processing environments.

Varying the Ratio of Tasks: In this experiment, we evaluated the performance of different methods in handling varying task ratios. The experiment set the ratios of image processing, log processing, and

data analysis to 2:1:1, 1:2:1, and 1:1:2. According to the results depicted in Fig. 5, the GD-DRL algorithm excelled in task scheduling, maintaining lower average response times and costs regardless of the task ratios, and also significantly outperformed other methods in terms of success rate.

Regarding average response time, when the task ratio was 2:1:1, all methods exhibited relatively high average response times. However, with the 1:1:2 task ratio, the average response time decreased slightly. This is attributed to the relatively slower processing speed of data analysis tasks, leading to increased response times as their proportion grew. As for overall cost, the methods showed relatively minor variations across different task ratios, indicating our approach maintained a relatively balanced resource allocation without significant cost increases due to specific task types. These results mean that task ratios influence algorithm performance, and our GD-DRL method demonstrates efficient and stable performance across different task ratios, maintaining low response times and costs.

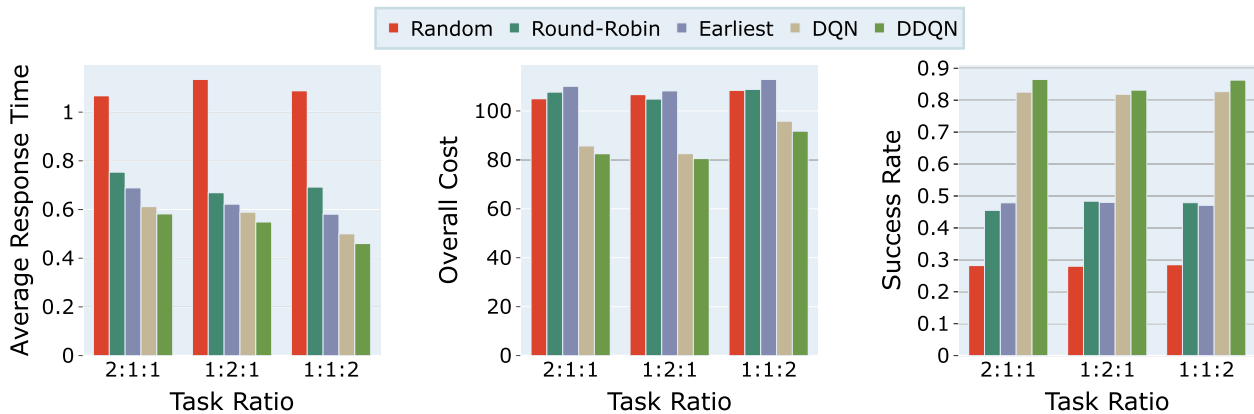


Fig. 5 Performance under different task ratios

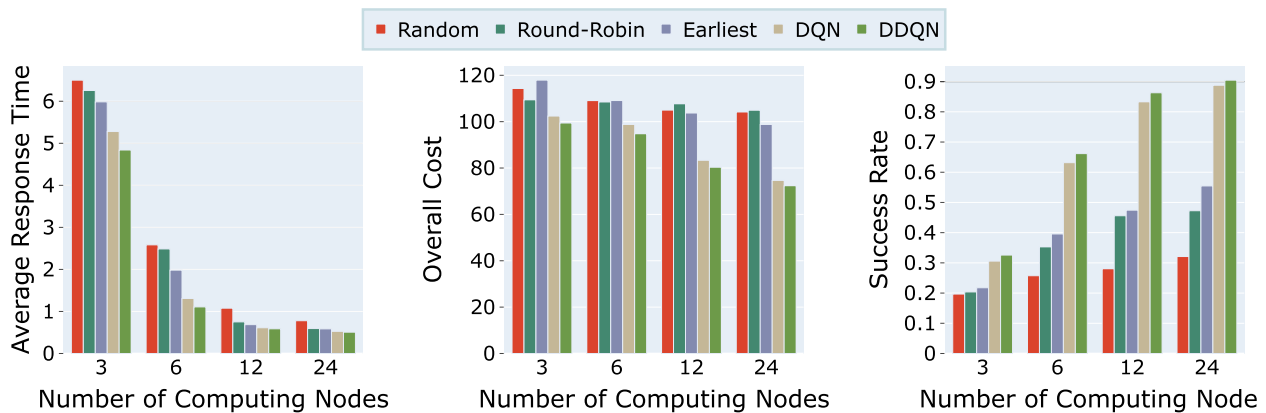


Fig. 6 Performance under different numbers of computing nodes

Varying the Number of Computing Nodes: In this experiment, we compared the performance of different methods in handling various tasks across different numbers of computing nodes. We set different numbers of computing nodes ranging from 3 to 24. According to the results depicted in Fig. 6, GD-DRL excelled in task scheduling, significantly outperforming other algorithms. Specifically, GD-DRL exhibited lower average response times and costs across different numbers of computing nodes, while also leading in success rate.

Response times showed a decline as the computational node count rose, with the DDQN outperforming others, particularly noticeable at higher node counts. Such observations imply that the influence of scheduling algorithms on response time becomes somewhat marginal when resources are scarce. However, the GD-DRL consistently achieved the lowest average response time, signifying its superior resource utilization efficiency. With an increase in the number of computational nodes, we observed a decrease in response time and an increase in success rate. As more computational nodes imply more resources available for selection, it can enhance the efficiency and success rate of task scheduling. These results show that, across different resource configurations, GD-DRL consistently outperforms other algorithms in terms of response time, cost, and success rate.

Conclusion

In this work, we propose a cloud computing architecture for power grid task processing, leveraging DT technology used in power grid operations. We also introduce GD-DRL, a novel real-time optimized task scheduling model designed to improve response time

and reduce costs at cloud computing nodes. By carefully considering the unique characteristics of tasks generated by the power grid and DT system, GD-DRL provides a robust solution for task scheduling. Our comprehensive design and empirical evaluations demonstrate that GD-DRL consistently outperforms other methods in task scheduling performance. In future work, we plan to extend GD-DRL to edge cloud environments. Decentralizing task processing closer to the data source will significantly improve response time and reduce latency. These improvements will be crucial for enhancing the resilience and efficiency of power grid operations, especially in dynamic environments.

Acknowledgements

Not applicable.

Authors' contributions

Daokun Qi: Conceptualization, Writing - original draft. Xiaojuan Xi: Methodology, Writing - review & editing. Yake Tang: Conceptualization, Methodology, Writing - review & editing. Yuesong Zhen: Methodology, Writing - review & editing. Zhenwei Guo: Methodology, Writing - review & editing.

Funding

This work was funded by the State Grid Henan Electric Power Company under the grant 5217L0230009.

Availability of data and materials

No datasets were generated or analysed during the current study.

Declarations

Ethics approval and consent to participate

Not applicable.

Consent for publication

Not applicable.

Competing interests

The authors declare no competing interests.

Received: 24 April 2024 Accepted: 24 June 2024

Published online: 08 July 2024

References

- Liu J, Wang Q, Song Z, Fang F (2021) Bottlenecks and countermeasures of high-penetration renewable energy development in China. *Engineering* 7(11):1611–1622
- Wang W, Liu J, Zeng D, Fang F, Niu Y (2020) Modeling and flexible load control of combined heat and power units. *Appl Therm Eng* 166:114624
- Fang F, Zhu Z, Jin S, Hu S (2020) Two-layer game theoretic microgrid capacity optimization considering uncertainty of renewable energy. *IEEE Syst J* 15(3):4260–4271
- Pan H, Dou Z, Cai Y, Li W, Lei X, Han D (2020) Digital twin and its application in power system. In: 2020 5th International Conference on Power and Renewable Energy (ICPRE). Shanghai, IEEE, pp 21–26
- Liu J, Song D, Li Q, Yang J, Hu Y, Fang F, Joo YH (2023) Life cycle cost modelling and economic analysis of wind power: A state of art review. *Energy Convers Manag* 277:116628
- Lu Y, Lv X, Fang F, Yang T, Romero CE (2020) Adaptive selective catalytic reduction model development using typical operating data in coal-fired power plants. *Energy* 192:116589
- Cheng L, Kotoulas S (2015) Efficient skew handling for outer joins in a cloud computing environment. *IEEE Trans Cloud Comput* 6(2):558–571
- Mao Y, Yan W, Song Y, Zeng Y, Chen M, Cheng L, Liu Q (2022) Differentiate quality of experience scheduling for deep learning inferences with docker containers in the cloud. *IEEE Trans Cloud Comput* 11(2):1667–1677
- Mao Y, Fu Y, Zheng W, Cheng L, Liu Q, Tao D (2021) Speculative container scheduling for deep learning applications in a kubernetes cluster. *IEEE Syst J* 16(3):3770–3781
- Liu Q, Xia T, Cheng L, Van Eijk M, Ozcelebi T, Mao Y (2021) Deep reinforcement learning for load-balancing aware network control in iot edge systems. *IEEE Trans Parallel Distrib Syst* 33(6):1491–1502
- Liu Q, Cheng L, Jia AL, Liu C (2021) Deep reinforcement learning for communication flow control in wireless mesh networks. *IEEE Netw* 35(2):112–119
- Cheng L, Wang Y, Cheng F, Liu C, Zhao Z, Wang Y (2023) A deep reinforcement learning-based preemptive approach for cost-aware cloud job scheduling. *IEEE Trans Sustain Comput* 9(3):422–432
- Zhang J, Cheng L, Liu C, Zhao Z, Mao Y (2023) Cost-aware scheduling systems for real-time workflows in cloud: An approach based on genetic algorithm and deep reinforcement learning. *Expert Syst Appl* 234:120972
- Chen Y, Gu W, Xu J, Zhang Y, Min G (2023) Dynamic task offloading for digital twin-empowered mobile edge computing via deep reinforcement learning. *China Commun* 20(11):164–175
- Consul P, Budhiraja I, Garg D, Kumar N, Singh R, Almogren AS (2024) A hybrid task offloading and resource allocation approach for digital twin-empowered uav-assisted mec network using federated reinforcement learning for future wireless network. *IEEE Trans Consum Electron*
- Durão LFC, Haag S, Anderl R, Schützer K, Zancul E (2018) Digital twin requirements in the context of industry 4.0. In: *Product Lifecycle Management to Support Industry 4.0: 15th IFIP WG 5.1 International Conference, PLM 2018, Turin, Italy, July 2–4, 2018, Proceedings 15*. Turin, Springer, pp 204–214
- Jeremiah SR, Yang LT, Park JH (2024) Digital twin-assisted resource allocation framework based on edge collaboration for vehicular edge computing. *Futur Gener Comput Syst* 150:243–254
- Khan SA, Rehman HZU, Waqar A, Khan ZH, Hussain M, Masud U (2023) Digital twin for advanced automation of future smart grid. In: 2023 1st International Conference on Advanced Innovations in Smart Cities (ICAISC). Jeddah, IEEE, pp 1–6
- Li W, Rentemeister M, Badeda J, Jöst D, Schulte D, Sauer DU (2020) Digital twin for battery systems: Cloud battery management system with online state-of-charge and state-of-health estimation. *J Energy Storage* 30:101557
- Liao H, Zhou Z, Liu N, Zhang Y, Xu G, Wang Z, Mumtaz S (2022) Cloud-edge-device collaborative reliable and communication-efficient digital twin for low-carbon electrical equipment management. *IEEE Trans Ind Inform* 19(2):1715–1724
- Chen X, Cheng L, Liu C, Liu Q, Liu J, Mao Y, Murphy J (2020) A woa-based optimization approach for task scheduling in cloud computing systems. *IEEE Syst J* 14(3):3117–3128
- Shukri SE, Al-Sayyed R, Hudaib A, Mirjalili S (2021) Enhanced multi-verse optimizer for task scheduling in cloud computing environments. *Expert Syst Appl* 168:114230
- Fang F, Wu X (2020) A win-win mode: The complementary and coexistence of 5g networks and edge computing. *IEEE Internet Things J* 8(6):3983–4003
- Jin S, Wang S, Fang F (2021) Game theoretical analysis on capacity configuration for microgrid based on multi-agent system. *Int J Electr Power Energy Syst* 125:106485
- Zade BMH, Mansouri N, Javidi MM (2021) SAEA: A security-aware and energy-aware task scheduling strategy by Parallel Squirrel Search Algorithm in cloud environment. *Expert Syst Appl* 176:114915
- Cho C, Shin S, Jeon H, Yoon S (2020) Qos-aware workload distribution in hierarchical edge clouds: A reinforcement learning approach. *IEEE Access* 8:193297–193313
- Zhang Y, Hu J, Min G (2023) Digital twin-driven intelligent task offloading for collaborative mobile edge computing. *IEEE J Sel Areas Commun* 41(10):3034–3045. <https://doi.org/10.1109/JSAC.2023.3310058>
- Xu X, Shen B, Ding S, Srivastava G, Bilal M, Khosravi MR, Menon VG, Jan MA, Wang M (2020) Service offloading with deep q-network for digital twinning-empowered internet of vehicles in edge computing. *IEEE Trans Ind Inform* 18(2):1414–1423
- Wang K, Yuan P, Jan MA, Khan F, Gadekallu TR, Kumari S, Pan H, Liu L (2024) Digital twin-assisted service function chaining in multi-domain computing power networks with multi-agent reinforcement learning. *Futur Gener Comput Syst* 158:294–307
- Zhu L, Tan L (2024) Task offloading scheme of vehicular cloud edge computing based on digital twin and improved a3c. *Internet Things* 26:101192
- Zhou Z, Jia Z, Liao H, Lu W, Mumtaz S, Guizani M, Tariq M (2021) Secure and latency-aware digital twin assisted resource scheduling for 5g edge computing-empowered distribution grids. *IEEE Trans Ind Inform* 18(7):4933–4943
- Gu Y, Cheng F, Yang L, Xu J, Chen X, Cheng L (2024) Cost-aware cloud workflow scheduling using drl and simulated annealing. *Digit Commun Netw*
- Chen X, Yu Q, Dai S, Sun P, Tang H, Cheng L (2023) Deep reinforcement learning for efficient iot data compression in smart railroad management. *IEEE Internet Things J*
- Abd Elaziz M, Attiya I (2021) An improved henry gas solubility optimization algorithm for task scheduling in cloud computing. *Artif Intell Rev* 54(5):3599–3637
- Guo H, Zhou X, Wang J, Liu J, Benslimane A (2023) Intelligent task offloading and resource allocation in digital twin based aerial computing networks. *IEEE J Sel Areas Commun* 41(10):3095–3110. <https://doi.org/10.1109/JSAC.2023.3310067>
- Ragazzini L, Negri E, Macchi M (2021) A digital twin-based predictive strategy for workload control. *IFAC-PapersOnLine* 54(1):743–748
- Altman E (2021) *Constrained Markov decision processes*. Boca Raton, Routledge
- Tong Z, Chen H, Deng X, Li K, Li K (2020) A scheduling scheme in the cloud computing environment using deep q-learning. *Inf Sci* 512:1170–1191
- Shyalika C, Silva T, Karunananda A (2020) Reinforcement learning in dynamic task scheduling: A review. *SN Comput Sci* 1(6):306
- Mnih V, Kavukcuoglu K, Silver D, Rusu AA, Veness J, Bellemare MG, Graves A, Riedmiller M, Fidjeland AK, Ostrovski G et al (2015) Human-level control through deep reinforcement learning. *Nature* 518(7540):529–533
- Van Hasselt H, Guez A, Silver D (2016) Deep reinforcement learning with double q-learning. In: *Proceedings of the AAAI conference on artificial intelligence*, vol 30. Arizona, AAAI Press

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.